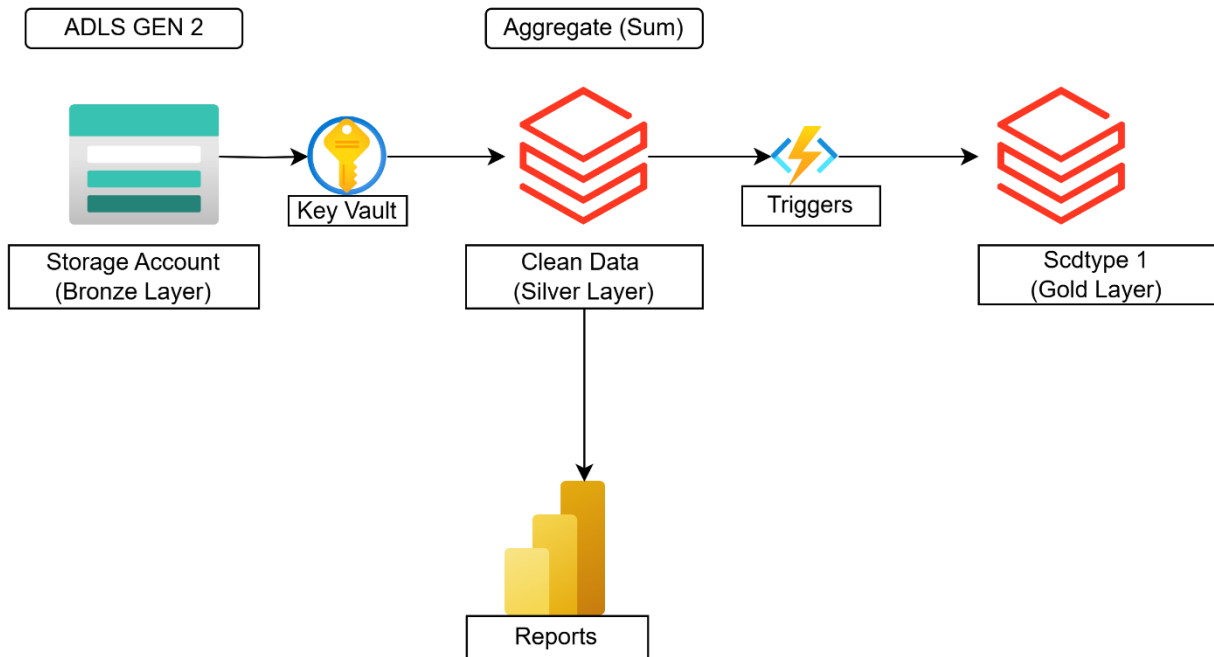


TRANSACTIONS AND LOAN DATA FOR A CUSTOMER

Objective

The project aims to design and implement a robust data pipeline for processing customer account data. This includes copying data from ADLS GEN2 (Bronze layer) and Transform the data in silver layer using Data bricks Notebooks and GOLD Storage into SCDType 1 Delta Table in ADLS GEN2. The pipeline aims to ensure efficient, accurate, and scalable data processing to support downstream analytics and reporting needs.

Medallion Architecture



BRONZE LAYER

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Location: bronze

Search blobs by prefix (case-sensitive)

☐ Show deleted objects

	Name	Modified	Access tier	Archive status	Blob type	Size	Lease state	
☐	accounts.csv	4/19/2025, 4:10:37 PM	Hot (Inferred)		Block blob	2.28 KiB	Available	...
☐	customers.csv	4/19/2025, 4:10:37 PM	Hot (Inferred)		Block blob	4.5 KiB	Available	...
☐	loan_payments.csv	4/19/2025, 4:10:37 PM	Hot (Inferred)		Block blob	2.55 KiB	Available	...
☐	loans.csv	4/19/2025, 4:10:37 PM	Hot (Inferred)		Block blob	2.29 KiB	Available	...
☐	transactions.csv	4/19/2025, 4:10:37 PM	Hot (Inferred)		Block blob	3.43 KiB	Available	...

SPN

Created services principles

to create service principle



```
▶ 10:28 AM (14s) 3

%python
configs = {
    "fs.azure.account.auth.type": "OAuth",
    "fs.azure.account.oauth.provider.type": "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
    "fs.azure.account.oauth2.client.id": dbutils.secrets.get(scope="projectconnection", key="app"),
    "fs.azure.account.oauth2.client.secret": dbutils.secrets.get(scope="projectconnection", key="secretval"),
    "fs.azure.account.oauth2.client.endpoint": "https://login.microsoftonline.com/0d6df8cc-1126-45dd-8615-8fde31f70319/oauth2/token"
}

dbutils.fs.mount(
    source="abfss://bronze@dhanvingen2.dfs.core.windows.net/",
    mount_point="/mnt/task",
    extra_configs=configs
)

True
```

Create the scope

```
▶ 10:20 AM (1s) 4

dbutils.secrets.listScopes()

[SecretScope(name='adlsgen2'),
 SecretScope(name='adlsgen2new'),
 SecretScope(name='gen2'),
 SecretScope(name='genconnection'),
 SecretScope(name='projectconnection')]
```

```
▶ 10:20 AM (1s) 5

dbutils.secrets.list("projectconnection")

[SecretMetadata(key='app'),
 SecretMetadata(key='projectkey'),
 SecretMetadata(key='secretval')]
```

SILVER LAYER

Transformation like removing nulls and duplicates in databricks

accounts

11:51 AM (10s) 8

```
df_acc = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load("/mnt/task/accounts.csv")
display(df_acc)
```

(3) Spark Jobs

df_acc: pyspark.sql.dataframe.DataFrame = [account_id: integer, customer_id: integer ... 2 more fields]

Table +

	account_id	customer_id	account_type	balance
1	1	45	Savings	1000.5
2	2	12	Checking	2500.75
3	3	78	Savings	1500
4	4	34	Checking	3000.25
5	5	56	Savings	500
6	6	23	Checking	1200.5
7	7	89	Savings	800.75

11:51 AM (2s) 9

```
df_acc.dropDuplicates().display()
```

(2) Spark Jobs

Table +

	account_id	customer_id	account_type	balance
1	36	27	Checking	3700
2	85	65	Savings	800.25
3	82	2	Checking	8300.5
4	52	10	Checking	5300
5	65	69	Savings	550.25
6	76	22	Checking	7700
7	62	35	Checking	6300.5
8	98	49	Checking	9900.5
9	53	86	Savings	400.25
10	66	26	Checking	6700.5
11	72	17	Checking	7300

▶

✓ 11:51 AM (1s)

10

Python

```
from pyspark.sql.functions import sum

grouped_df = df_acc.groupBy("account_type").agg(
    sum("balance").alias("total_amount")
)

grouped_df.show()
```

▶ (2) Spark Jobs

▶ grouped_df: pyspark.sql.dataframe.DataFrame = [account_type: string, total_amount: double]

+-----+-----+
|account_type|total_amount|
+-----+-----+
| Savings| 28799.25|
| Checking| 269913.0|
+-----+-----+

▶

✓ 11:51 AM (5s)

11

```
%python
df_acc.write.format("parquet").mode("overwrite").save('/mnt/container/silverdatabricks/accounts')
```

▶ (1) Spark Jobs

customers

▶

✓ 11:51 AM (1s)

13

```
df_cus = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load("/mnt/task/customers.csv")
```

▶ (2) Spark Jobs

▶ df_cus: pyspark.sql.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

▶

✓ 11:51 AM (<1s)

14

```
display(df_cus)
```

▶ (1) Spark Jobs

Table

+

	customer_id	first_name	last_name	address	city	state	zip
1	1	John	Doe	123 Elm St	Toronto	ON	M4B1B3
2	2	Jane	Smith	456 Maple Ave	Ottawa	ON	K1A0B1
3	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1
4	4	Emily	Davis	101 Pine Rd	Calgary	AB	T2A0A1
5	5	David	Wilson	202 Birch Blvd	Vancouver	BC	V5K0A1
6	6	Emma	Clark	505 Cedar St	Halifax	NS	B3H0A1

rename nulls

```
11:51 AM (<1s) 16
df_cleaned = df_cus.na.fill({"state": "unknown", "zip": "unknown"})

display(df_cleaned)
```

(1) Spark Jobs

df_cleaned: pyspark.sql.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

	¹ ₃ customer_id	^A _C first_name	^A _C last_name	^A _C address	^A _C city	^A _C state	^A _C zip
1	1	John	Doe	123 Elm St	Toronto	ON	M4B1B3
2	2	Jane	Smith	456 Maple Ave	Ottawa	ON	K1A0B1
3	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1
4	4	Emily	Davis	101 Pine Rd	Calgary	AB	T2A0A1
5	5	David	Wilson	202 Birch Blvd	Vancouver	BC	V5K0A1
6	6	Emma	Clark	505 Cedar St	Halifax	NS	B3H0A1

```
11:51 AM (1s) 17
df_cus.dropDuplicates().display()
```

(2) Spark Jobs

	¹ ₃ customer_id	^A _C first_name	^A _C last_name	^A _C address	^A _C city	^A _C state	^A _C zip
1	32	Sophia	Morris	3131 Oak Dr	Belleville	ON	K8N0A1
2	34	Olivia	Reed	3333 Birch Blvd	Orillia	ON	L3V0A1
3	25	Daniel	Campbell	2424 Willow Rd	St. Catharines	ON	L2R0A1
4	43	Joseph	Cox	4242 Cedar Ln	Aurora	ON	L4G0A1
5	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1
6	19	Christopher	Baker	1818 Pine Rd	Thunder Bay	ON	P7A0A1
7	76	Evelyn	Wallace	7575 Birch Blvd	Huntsville	ON	P1H0A1
8	11	Alexander	Thomas	1010 Willow Rd	St. John's	NL	A1A0A1

```
11:51 AM (3s) 18
%python
df_cleaned.write.format("parquet").mode("overwrite").save('/mnt/container/silverdatabricks/customers')
```

(1) Spark Jobs

loan_payments

▶ 11:51 AM (1s) 20

```
df_lopay = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load("/mnt/task/loan_payments.csv")
display(df_lopay)
```

▶ (3) Spark Jobs

▶ df_lopay: pyspark.sql.dataframe.DataFrame = [payment_id: integer, loan_id: integer ... 2 more fields]

Table + 🔍 ⚙️ □

	¹ / ₃ payment_id	¹ / ₃ loan_id	payment_date	1.2 payment_amount
1	1	45	2024-01-01	100
2	2	23	2024-01-02	150
3	3	67	2024-01-03	200
4	4	89	2024-01-04	250
5	5	12	2024-01-05	300

▶ 11:51 AM (1s) 21

```
df_lopay.dropDuplicates().display()
```

▶ (2) Spark Jobs

Table + 🔍 ⚙️ □

	¹ / ₃ payment_id	¹ / ₃ loan_id	payment_date	1.2 payment_amount
1	13	44	2024-01-13	700
2	32	53	2024-02-01	1650
3	93	24	2024-04-02	4700
4	76	37	2024-03-16	3850
5	30	31	2024-01-30	1550
6	67	38	2024-03-07	3400

▶ 11:51 AM (2s) 22

```
%python
df_lopay.write.format("parquet").mode("overwrite").save('/mnt/container/silverdatabricks/loan_payments')
```

▶ (1) Spark Jobs

loans

▶ ✓ 11:51 AM (1s) 24

df_lo = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load("/mnt/task/loans.csv")
display(df_lo)

▶ (3) Spark Jobs

▶ df_lo: pyspark.sql.dataframe.DataFrame = [loan_id: integer, customer_id: integer ... 3 more fields]

Table ▾ +

🔍 🏠 📄 🗑️

	^{1 2} loan_id	^{1 2} customer_id	^{1 2} loan_amount	^{1 2} interest_rate	^{1 2} loan_term
1	1	45	10000.5	5.5	36
2	2	12	20000.75	4.5	48
3	3	78	15000	6	60
4	4	34	30000.25	3.5	24
5	5	56	25000	5	36
6	6	23	17500.5	4	48
7	7	89	22500.75	6.5	60

▶ ✓ 11:51 AM (1s) 25

df_lo.dropDuplicates().display()

▶ (2) Spark Jobs

Table ▾ +

🔍 🏠 📄 🗑️

	^{1 2} loan_id	^{1 2} customer_id	^{1 2} loan_amount	^{1 2} interest_rate	^{1 2} loan_term
1	5	56	25000	5	36
2	84	40	30000	3.5	24
3	87	93	22500.75	6.5	60
4	51	72	10000.75	6	60
5	68	8	27500	3.5	24
6	95	60	25000.75	6.5	60
7	57	97	22500.25	5.5	36

▶ ✓ 11:51 AM (2s) 26

%python
df_lo.write.format("parquet").mode("overwrite").save('/mnt/container/silverdatabricks/loans')

▶ (1) Spark Jobs

transactions

▶ 11:51 AM (1s)28

```
df_tra = spark.read.format("csv").option("header", "true").option("inferSchema", "true").load("/mnt/task/transactions.csv")
display(df_tra)
```

▶ (3) Spark Jobs

df_tra: pyspark.sql.dataframe.DataFrame = [transaction_id: integer, account_id: integer ... 3 more fields]

Table +

🔍🔼📄🗑

	123 transaction_id	123 account_id	📅 transaction_date	1.2 transaction_amount	ABC transaction_type
1	1	45	2024-01-01	100.5	Deposit
2	2	12	2024-01-02	200.75	Withdrawal
3	3	78	2024-01-03	150	Deposit

▶ 11:51 AM (1s)29

```
df_tra.dropDuplicates().display()
```

▶ (2) Spark Jobs

Table +

🔍🔼📄🗑

	123 transaction_id	123 account_id	📅 transaction_date	1.2 transaction_amount	ABC transaction_type
1	32	9	2024-02-01	200.75	Withdrawal
2	11	3	2024-01-11	100.5	Deposit
3	25	66	2024-01-25	250	Deposit
4	10	92	2024-01-10	375.25	Withdrawal
5	30	32	2024-01-30	375.25	Withdrawal
6	61	52	2024-03-01	100.5	Deposit
7	97	90	2024-04-06	225.5	Deposit

▶ 11:51 AM (2s)30Python🔼📄🗑

```
%python
df_tra.write.format("parquet").mode("overwrite").save('/mnt/container/silverdatabricks/transactions')
```

▶ (1) Spark Jobs

Here is the gen 2 storage account container which all the files have been loaded

Authentication method: Access key (Switch to Microsoft Entra user account)
Location: bronze / silverdatabricks

Search blobs by prefix (case-sensitive)

Show deleted objects

Name	Modified	Access tier	Archive status	Blob type	Size
☐  [-]					
☐  accounts	4/24/2025, 11:51:36 AM				
☐  combined	4/24/2025, 11:00:49 AM				
☐  customers	4/24/2025, 11:51:41 AM				
☐  loan_payments	4/24/2025, 11:51:46 AM				
☐  loans	4/24/2025, 11:51:50 AM				
☐  transactions	4/24/2025, 11:51:54 AM				

GOLD LAYER

scdtype 1 for all files

accounts scdtype1

▶

✓ 11:51 AM (<1s)

33

```
%python
dbutils.fs.ls('/mnt/task/silverdatabricks/accounts')
```

```
[FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_SUCCESS', name='_SUCCESS', size=0, modificationTime=1745509896000),
  FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_committed_236769075193495685', name='_committed_236769075193495685', size=230, modificationTime=1745509895000),
  FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_committed_5894471020512476432', name='_committed_5894471020512476432', size=123, modificationTime=1745504533000),
  FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_committed_vacuum677266064837893943', name='_committed_vacuum677266064837893943', size=95, modificationTime=1745509896000),
  FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_started_236769075193495685', name='_started_236769075193495685', size=0, modificationTime=1745509893000),
  FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/part-00000-tid-236769075193495685-75c466fc-35fb-446a-83f0-2f2161745981-7-1-c000.snappy.parquet', name='part-00000-tid-236769075193495685-75c466fc-35fb-446a-83f0-2f2161745981-7-1-c000.snappy.parquet', size=2804, modificationTime=1745509895000)]
```

▶ ✓ 11:51 AM (<1s) 34

```
%sql
use catalog hive_metastore
```

OK

+ Code + Text

▶ ✓ 11:51 AM (6s) 35

```
%sql
CREATE TABLE IF NOT EXISTS account_databricks (
  account_id INT,
  customer_id INT,
  account_type STRING,
  balance INT,
  hashkey BIGINT,
  createddate TIMESTAMP,
  createdby STRING,
  updateby STRING,
  updateddate TIMESTAMP
)
USING DELTA
LOCATION "/mnt/task/gold/account_databricks";
```

OK

▶ ✓ 11:51 AM (<1s) 36

```
from pyspark.sql.functions import *
```

▶ ✓ 11:51 AM (<1s) 37

```
src_path = '/mnt/task/silverdatabricks/accounts/'
dbutils.fs.ls(src_path)
```

```
[FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_SUCCESS', name='_SUCCESS', size=0, modificationTime=1745509896000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_committed_236769075193495685', name='_committed_236769075193495685', size=230, modificationTime=1745509895000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_committed_5894471020512476432', name='_committed_5894471020512476432', size=123, modificationTime=1745504533000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_committed_vacuum677266064837893943', name='_committed_vacuum677266064837893943', size=95, modificationTime=1745509896000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/_started_236769075193495685', name='_started_236769075193495685', size=0, modificationTime=1745509893000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/accounts/part-00000-tid-236769075193495685-75c466fc-35fb-446a-83f0-2f2161745981-7-1-c000.snappy.parquet', name='part-00000-tid-236769075193495685-75c466fc-35fb-446a-83f0-2f2161745981-7-1-c000.snappy.parquet', size=2804, modificationTime=1745509895000)]
```

```
▶ 11:51 AM (<1s) 38 Python

%python
tgt_path = '/mnt/task/gold/account_databricks'
dbutils.fs.ls(tgt_path)

[FileInfo(path='dbfs:/mnt/task/gold/account_databricks/_delta_log/', name='_delta_log/', size=0, modificationTime=1745504964000),
FileInfo(path='dbfs:/mnt/task/gold/account_databricks/part-00000-ea42b89d-cd6f-4e69-8737-b7e5d86b35a8-c000.snappy.parquet', name='part-00000-ea42b89d-cd6f-4e69-8737-b7e5d86b35a8-c000.snappy.parquet', size=4873, modificationTime=1745505012000)]
```

```
▶ 11:51 AM (2s) 39

df_acc1 = spark.read.format("parquet").option("header", "true").option("inferSchema", "true").load(src_path)
display(df_acc1)

(2) Spark Jobs

df_acc1: pyspark.sql.dataframe.DataFrame = [account_id: integer, customer_id: integer ... 2 more fields]
```

	123 account_id	123 customer_id	ABC account_type	1.2 balance
1	1	45	Savings	1000.5
2	2	12	Checking	2500.75
3	3	78	Savings	1500
4	4	34	Checking	3000.25

```
▶ 11:51 AM (<1s) 40

%python
from pyspark.sql.functions import hash, concat

df1_hash = df_acc1.withColumn("hashkey", hash(concat(*df_acc1.columns)))
display(df1_hash)

(1) Spark Jobs

df1_hash: pyspark.sql.dataframe.DataFrame = [account_id: integer, customer_id: integer ... 3 more fields]
```

	123 account_id	123 customer_id	ABC account_type	1.2 balance	123 hashkey
1	1	45	Savings	1000.5	949148725
2	2	12	Checking	2500.75	-230508604
3	3	78	Savings	1500	-1072894276
4	4	34	Checking	3000.25	-865923237
5	5	56	Savings	500	-1227076413
6	6	23	Checking	1200.5	-1550611064
7	7	89	Savings	800.75	1935814550
8	8	67	Checking	2200	1739753395
9	9	14	Savings	900.25	-1267919322

41

- ▶ (2) Spark Jobs

Table									
	1 ² ₃ account_id	1 ² ₃ customer_id	A ^B _C account_type	1 ² ₃ balance	1 ² ₃ hashkey	📅 createddate	A ^B _C cr		
1	1	45	Savings	1000	949148725	2025-04-24T14:30:11.616+00:...	data		
2	2	12	Checking	2500	-230508604	2025-04-24T14:30:11.616+00:...	data		
3	3	78	Savings	1500	-1072894276	2025-04-24T14:30:11.616+00:...	data		
4	4	34	Checking	3000	-865923237	2025-04-24T14:30:11.616+00:...	data		
5	5	56	Savings	500	-1227076413	2025-04-24T14:30:11.616+00:...	data		
6	6	23	Checking	1200	-1550611064	2025-04-24T14:30:11.616+00:...	data		
7	7	89	Savings	800	1935814550	2025-04-24T14:30:11.616+00:...	data		
8	8	67	Checking	2200	1739753395	2025-04-24T14:30:11.616+00:...	data		
9	9	14	Savings	900	-1267919322	2025-04-24T14:30:11.616+00:...	data		

42

- ▶ (2) Spark Jobs

1^2_3 account_id	1^2_3 customer_id	A^B_C account_type	1.2 balance	1^2_3 hashkey
--------------------	---------------------	----------------------	-------------	-----------------

No rows returned

▶

✓ 11:51 AM (12s)

43

```
%python
from pyspark.sql.functions import col, lit, current_timestamp

dtable.alias("tgt").merge(
    df_src1.alias("src"), col("src.account_id") == col("tgt.account_id"))\
        .whenMatchedUpdate(set={
            "account_id": col("src.account_id"),
            "customer_id": col("src.customer_id"),
            "account_type": col("src.account_type"),
            "balance": col("src.balance"),
            "hashkey": col("src.hashkey"),
            "updatedate": current_timestamp(),
            "updateby": lit("databricks-update")
        })\
        .whenNotMatchedInsert(values={
            "account_id": col("src.account_id"),
            "customer_id": col("src.customer_id"),
            "account_type": col("src.account_type"),
            "balance": col("src.balance"),
            "hashkey": col("src.hashkey"),
            "createddate": current_timestamp(),
            "updatedate": current_timestamp(),
            "createdby": lit("databricks"),
            "updateby": lit("databricks")
        })
).execute()
```

▶ (12) Spark Jobs

▶

✓ 11:51 AM (2s)

44

SQL

✦

⌵

⋮

```
%sql
select * from hive_metastore.default.account_databricks
```

▶ (2) Spark Jobs

▶ 📄 _sqldf: pyspark.sql.dataframe.DataFrame = [account_id: integer, customer_id: integer ... 7 more fields]

Table							
	1 ² account_id	1 ² customer_id	A ^B account_type	1 ² balance	1 ² hashkey	📅 createddate	A ^B ci
1	1	45	Savings	1000	949148725	2025-04-24T14:30:11.616+00:...	data:
2	2	12	Checking	2500	-230508604	2025-04-24T14:30:11.616+00:...	data:
3	3	78	Savings	1500	-1072894276	2025-04-24T14:30:11.616+00:...	data:
4	4	34	Checking	3000	-865923237	2025-04-24T14:30:11.616+00:...	data:
5	5	56	Savings	500	-1227076413	2025-04-24T14:30:11.616+00:...	data:
6	6	23	Checking	1200	-1550611064	2025-04-24T14:30:11.616+00:...	data:
7	7	89	Savings	800	1935814550	2025-04-24T14:30:11.616+00:...	data:
8	8	67	Checking	2200	1739753395	2025-04-24T14:30:11.616+00:...	data:
9	9	14	Savings	900	-1267919322	2025-04-24T14:30:11.616+00:...	data:
10	10	92	Checking	1800	-415585298	2025-04-24T14:30:11.616+00:...	data:

scdtype for customer

Markdown



+ Code + Text

▶ ✓ 11:51 AM (<1s)

46

```
%sql
use catalog hive_metastore
```

OK

▶ ✓ 11:51 AM (<1s)

47

```
%sql
CREATE TABLE IF NOT EXISTS customer_databricks (
  customer_id int,
  first_name string,
  last_name string,
  address string,
  city string,
  state string,
  zip string,
  hashkey BIGINT,
  createddate TIMESTAMP,
  createdby STRING,
  updateby STRING,
  updateddate TIMESTAMP
)
USING DELTA
LOCATION "/mnt/task/gold/customer_databricks";
```

▶ ✓ 11:51 AM (<1s)

48

Python



```
src_path = '/mnt/task/silverdatabricks/customers/'
dbutils.fs.ls(src_path)
```

```
[FileInfo(path='dbfs:/mnt/task/silverdatabricks/customers/_SUCCESS', name='_SUCCESS', size=0, modificationTime=1745509901000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/customers/_committed_3233297277930428266', name='_committed_3233297277930428266', size=124, modificationTime=1745506320000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/customers/_committed_8024598406090703730', name='_committed_8024598406090703730', size=233, modificationTime=1745509900000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/customers/_committed_vacuum4511946457831871607', name='_committed_vacuum4511946457831871607', size=96, modificationTime=1745509902000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/customers/_started_8024598406090703730', name='_started_8024598406090703730', size=0, modificationTime=1745509899000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/customers/part-00000-tid-8024598406090703730-65d94127-805d-47d3-94b7-cba9d142c459-14-1-c000.snappy.parquet', name='part-00000-tid-8024598406090703730-65d94127-805d-47d3-94b7-cba9d142c459-14-1-c000.snappy.parquet', size=5895, modificationTime=1745509900000)]
```

▶

✓

11:51 AM (<1s)

49

Python

✦

⌵

⋮

```
tgt_path = '/mnt/task/gold/customer_databricks'
dbutils.fs.ls(tgt_path)
```

[FileInfo(path='dbfs:/mnt/task/gold/customer_databricks/_delta_log/', name='_delta_log/', size=0, modificationTime=1745505032000),
FileInfo(path='dbfs:/mnt/task/gold/customer_databricks/part-00000-64f9e3e2-a1a6-40e6-8b3a-5c679774bb14-c000.snappy.parquet', name='part-00000-64f9e3e2-a1a6-40e6-8b3a-5c679774bb14-c000.snappy.parquet', size=7981, modificationTime=1745505072000)]

▶

✓

11:51 AM (1s)

50

```
df_customer = spark.read.format("parquet").option("header", "true").option("inferSchema", "true").load(src_path)
display(df_customer)
```

▶ (2) Spark Jobs

▶ df_customer: pyspark.sql.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

Table

▼

+

🔍

🔼

🔽

🔍

🔼

🔽

	customer_id	first_name	last_name	address	city	state	zip
1	1	John	Doe	123 Elm St	Toronto	ON	M4B1B3
2	2	Jane	Smith	456 Maple Ave	Ottawa	ON	K1A0B1
3	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1
4	4	Emily	Davis	101 Pine Rd	Calgary	AB	T2A0A1
5	5	David	Wilson	202 Birch Blvd	Vancouver	BC	V5K0A1
6	6	Emma	Clark	505 Cedar St	Halifax	NS	B3H0A1
7	7	James	Martinez	606 Spruce Ln	Winnipeg	MB	R3C0A1

▶

✓

11:51 AM (<1s)

51

Python

✦

⌵

⋮

```
%python
from pyspark.sql.functions import crc32, concat

df2_hash = df_customer.withColumn("hashkey", crc32(concat(*df_customer.columns)))
display(df2_hash)
```

▶ (1) Spark Jobs

▶ df2_hash: pyspark.sql.dataframe.DataFrame = [customer_id: integer, first_name: string ... 6 more fields]

Table

▼

+

🔍

🔼

🔽

🔍

🔼

🔽

	customer_id	first_name	last_name	address	city	state	zip	hashkey
1	1	John	Doe	123 Elm St	Toronto	ON	M4B1B3	2373
2	2	Jane	Smith	456 Maple Ave	Ottawa	ON	K1A0B1	3911
3	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1	4114
4	4	Emily	Davis	101 Pine Rd	Calgary	AB	T2A0A1	165
5	5	David	Wilson	202 Birch Blvd	Vancouver	BC	V5K0A1	3284
6	6	Emma	Clark	505 Cedar St	Halifax	NS	B3H0A1	4268
7	7	James	Martinez	606 Spruce Ln	Winnipeg	MB	R3C0A1	4278
8	8	Olivia	Garcia	707 Fir St	Edmonton	AB	T5A0A1	443

▶

✓ 11:51 AM (1s)

52

Python

✚

⌵

⋮

```
%python
from delta.tables import DeltaTable
dtable1 = DeltaTable.forPath(spark, tgt_path)
display(dtable1.toDF())
```

▶ (2) Spark Jobs

Table ▾ +

🔍

🏠

📄

🗨

	customer_id	first_name	last_name	address	city	state	zip	hashke
1	1	John	Doe	123 Elm St	Toronto	ON	M4B1B3	2373
2	2	Jane	Smith	456 Maple Ave	Ottawa	ON	K1A0B1	3911
3	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1	4114
4	4	Emily	Davis	101 Pine Rd	Calgary	AB	T2A0A1	165
5	5	David	Wilson	202 Birch Blvd	Vancouver	BC	V5K0A1	3284
6	6	Emma	Clark	505 Cedar St	Halifax	NS	B3H0A1	4266
7	7	James	Martinez	606 Spruce Ln	Winnipeg	MB	R3C0A1	4276
8	8	Olivia	Garcia	707 Fir St	Edmonton	AB	T5A0A1	443
9	9	William	Lopez	808 Redwood Dr	Victoria	BC	V8W0A1	4186

▶

✓ 11:51 AM (1s)

53

```
%python
from pyspark.sql.functions import col

df_src2 = df2_hash.alias("src").join(
    dtable1.toDF().alias("tgt"),
    ((col("src.customer_id") == col("tgt.customer_id")) & (col("src.hashkey") == col("tgt.hashkey"))),
    "anti").select("src.*")

display(df_src2)
```

▶ (2) Spark Jobs

df_src2: pyspark.sql.dataframe.DataFrame = [customer_id: integer, first_name: string ... 6 more fields]

Table ▾ +

🔍

🏠

📄

🗨

	customer_id	first_name	last_name	address	city	state	zip	hashkey
1	87	William	McDonald	8686 Maple Ave	Haileybury	unknown	unknown	397430582

```

dtable1.alias("tgt").merge(
  df_src2.alias("src"), col("src.customer_id") == col("tgt.customer_id"))\
  .whenMatchedUpdate(set={
    "customer_id": col("src.customer_id"),
    "first_name": col("src.first_name"),
    "last_name": col("src.last_name"),
    "address": col("src.address"),
    "city": col("src.city"),
    "state": col("src.state"),
    "zip": col("src.zip"),
    "hashkey": col("src.hashkey"),
    "updatedate": current_timestamp(),
    "updateby": lit("databricks-update")
  })\
  .whenNotMatchedInsert(values={
    "customer_id": col("src.customer_id"),
    "first_name": col("src.first_name"),
    "last_name": col("src.last_name"),
    "address": col("src.address"),
    "city": col("src.city"),
    "state": col("src.state"),
    "zip": col("src.zip"),
    "hashkey": col("src.hashkey"),
    "createddate": current_timestamp(),
    "updatedate": current_timestamp(),
    "createdby": lit("databricks"),
    "updateby": lit("databricks")
  })
).execute()

```

11:51 AM (2s) 55

```
%sql
select * from hive_metastore.default.customer_databricks
```

(3) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [customer_id: integer, first_name: string ... 10 more fields]

	customer_id	first_name	last_name	address	city	state	zip	hashke
1	1	John	Doe	123 Elm St	Toronto	ON	M4B1B3	2373
2	2	Jane	Smith	456 Maple Ave	Ottawa	ON	K1A0B1	3911
3	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1	4114
4	4	Emily	Davis	101 Pine Rd	Calgary	AB	T2A0A1	165
5	5	David	Wilson	202 Birch Blvd	Vancouver	BC	V5K0A1	3284
6	6	Emma	Clark	505 Cedar St	Halifax	NS	B3H0A1	4268
7	7	James	Martinez	606 Spruce Ln	Winnipeg	MB	R3C0A1	4278
8	8	Olivia	Garcia	707 Fir St	Edmonton	AB	T5A0A1	443
9	9	William	Lopez	808 Redwood Dr	Victoria	BC	V8W0A1	4188
10	10	Ava	Anderson	909 Cypress Ave	Quebec City	QC	G1A0A1	980
11	11	Alexander	Thomas	1010 Willow Rd	St. John's	NL	A1A0A1	823

loan_payments

▶ ✓ 11:51 AM (<1s)

57

```
%sql
use catalog hive_metastore
```

OK

▶ ✓ 11:51 AM (<1s)

58

```
%sql
CREATE TABLE IF NOT EXISTS loan_payment_databricks (
  payment_id int,
  loan_id int,
  payment_date date,
  payment_amount int,
  hashkey BIGINT,
  createddate TIMESTAMP,
  createdby STRING,
  updateby STRING,
  updateddate TIMESTAMP
)
USING DELTA
LOCATION "/mnt/task/gold/loan_payment_databricks";
```

OK

▶ ✓ 11:51 AM (<1s)

59

Python ✨ ⌂ ⋮

```
src_path = '/mnt/task/silverdatabricks/loan_payments/'
dbutils.fs.ls(src_path)
```

```
[FileInfo(path='dbfs:/mnt/task/silverdatabricks/loan_payments/_SUCCESS', name='_SUCCESS', size=0, modificationTime=1745509906000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loan_payments/_committed_2473684866283077217', name='_committed_2473684866283077217', size=124, modificationTime=1745504576000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loan_payments/_committed_7091392688889552215', name='_committed_7091392688889552215', size=233, modificationTime=1745509905000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loan_payments/_committed_vacuum1618860441328294764', name='_committed_vacuum1618860441328294764', size=96, modificationTime=1745509906000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loan_payments/_started_7091392688889552215', name='_started_7091392688889552215', size=0, modificationTime=1745509905000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loan_payments/part-00000-tid-7091392688889552215-42831fb3-5047-4207-b78d-c64fd8cd957f-20-1-c000.snappy.parquet', name='part-00000-tid-7091392688889552215-42831fb3-5047-4207-b78d-c64fd8cd957f-20-1-c000.snappy.parquet', size=3140, modificationTime=1745509905000)]
```

```
11:51 AM (<1s) 60

tgt_path = '/mnt/task/gold/loan_payment_databricks'
dbutils.fs.ls(tgt_path)

[FileInfo(path='dbfs:/mnt/task/gold/loan_payment_databricks/_delta_log/', name='_delta_log/', size=0, modificationTime=1745505109000),
 FileInfo(path='dbfs:/mnt/task/gold/loan_payment_databricks/part-00000-776ea8e7-561e-43c5-820d-5cdb0d83ae2d-c000.snappy.parquet', name='part-00000-776ea8e7-561e-43c5-820d-5cdb0d83ae2d-c000.snappy.parquet', size=5257, modificationTime=1745505143000)]
```

11:51 AM (1s) 61

df_loan_payment = spark.read.format("parquet").load(src_path)
display(df_loan_payment)

(2) Spark Jobs

df_loan_payment: pyspark.sql.dataframe.DataFrame = [payment_id: integer, loan_id: integer ... 2 more fields]

Table +

	1 payment_id	1 loan_id	1 payment_date	1.2 payment_amount
1	1	45	2024-01-01	100
2	2	23	2024-01-02	150
3	3	67	2024-01-03	200
4	4	89	2024-01-04	250
5	5	12	2024-01-05	300

11:51 AM (<1s) 62

df3_hask = df_loan_payment.withColumn("hashkey", crc32(concat(*df_loan_payment.columns)))
display(df3_hask)

(1) Spark Jobs

df3_hask: pyspark.sql.dataframe.DataFrame = [payment_id: integer, loan_id: integer ... 3 more fields]

Table +

	1 payment_id	1 loan_id	1 payment_date	1.2 payment_amount	1 hashkey
1	1	45	2024-01-01	100	4189237233
2	2	23	2024-01-02	150	1457204963
3	3	67	2024-01-03	200	2777852429
4	4	89	2024-01-04	250	264335070
5	5	12	2024-01-05	300	438074744
6	6	34	2024-01-06	350	3330136997
7	7	56	2024-01-07	400	1192820904
8	8	78	2024-01-08	450	2203091019

11:51 AM (1s)

63

```
%python
from delta.tables import DeltaTable
dtable3 = DeltaTable.forPath(spark, tgt_path)
display(dtable3.toDF())
```

(2) Spark Jobs

Table +

🔍 🗑️ 📄 🗑️

	¹ ₃ payment_id	¹ ₃ loan_id	payment_date	¹ ₃ payment_amount	¹ ₃ hashkey	createddate
1	1	45	2024-01-01	100	4189237233	2025-04-24T14:32:21.938+00:...
2	2	23	2024-01-02	150	1457204963	2025-04-24T14:32:21.938+00:...
3	3	67	2024-01-03	200	2777852429	2025-04-24T14:32:21.938+00:...
4	4	89	2024-01-04	250	264335070	2025-04-24T14:32:21.938+00:...
5	5	12	2024-01-05	300	438074744	2025-04-24T14:32:21.938+00:...
6	6	34	2024-01-06	350	3330136997	2025-04-24T14:32:21.938+00:...
7	7	56	2024-01-07	400	1192820904	2025-04-24T14:32:21.938+00:...
8	8	78	2024-01-08	450	2203091019	2025-04-24T14:32:21.938+00:...

11:51 AM (1s)

64

```
df_src3 =df3_hask.alias("src").join(dtable3.toDF().alias("tgt"),((col("src.payment_id") == col("tgt.payment_id")) & (col
("src.hashkey") == col("tgt.hashkey"))), "anti").select("src.*")
display(df_src3)
```

(2) Spark Jobs

df_src3: pyspark.sql.dataframe.DataFrame = [payment_id: integer, loan_id: integer ... 3 more fields]

Table +

🔍 🗑️ 📄 🗑️

¹ ₃ payment_id	¹ ₃ loan_id	payment_date	¹ ₂ payment_amount	¹ ₃ hashkey
--------------------------------------	-----------------------------------	--------------	--	-----------------------------------

No rows returned

▶

✓

11:51 AM (4s)

65

Python

✦

⌵

⋮

```
%python
from pyspark.sql.functions import col, lit, current_timestamp

dtable3.alias("tgt").merge(
    df_src3.alias("src"),col("src.payment_id") == col("tgt.payment_id"))\
        .whenMatchedUpdate(set={
            "payment_id": col("src.payment_id"),
            "loan_id": col("src.loan_id"),
            "payment_date": col("src.payment_date"),
            "payment_amount": col("src.payment_amount"),
            "hashkey": col("src.hashkey"),
            "updatedate": current_timestamp(),
            "updateby": lit("databricks-update")
        })\
        .whenNotMatchedInsert(values={
            "payment_id": col("src.payment_id"),
            "loan_id": col("src.loan_id"),
            "payment_date": col("src.payment_date"),
            "payment_amount": col("src.payment_amount"),
            "hashkey": col("src.hashkey"),
            "createddate": current_timestamp(),
            "updatedate": current_timestamp(),
            "createdby": lit("databricks"),
            "updateby": lit("databricks")
        })
).execute()
```

▶ (12) Spark Jobs

▶

✓

11:51 AM (1s)

66

```
%sql
select * from hive_metastore.default.loan_payment_databricks
```

▶ (2) Spark Jobs

▶ _sqldf: pyspark.sql.dataframe.DataFrame = [payment_id: integer, loan_id: integer ... 7 more fields]

	1²₃ payment_id	1²₃ loan_id	📅 payment_date	1²₃ payment_amount	1²₃ hashkey	📅 createddate
1	1	45	2024-01-01	100	4189237233	2025-04-24T14:32:21.938+00:...
2	2	23	2024-01-02	150	1457204963	2025-04-24T14:32:21.938+00:...
3	3	67	2024-01-03	200	2777852429	2025-04-24T14:32:21.938+00:...
4	4	89	2024-01-04	250	264335070	2025-04-24T14:32:21.938+00:...
5	5	12	2024-01-05	300	438074744	2025-04-24T14:32:21.938+00:...
6	6	34	2024-01-06	350	3330136997	2025-04-24T14:32:21.938+00:...
7	7	56	2024-01-07	400	1192820904	2025-04-24T14:32:21.938+00:...
8	8	78	2024-01-08	450	2203091019	2025-04-24T14:32:21.938+00:...
9	9	90	2024-01-09	500	2681690964	2025-04-24T14:32:21.938+00:...
10	10	11	2024-01-10	550	3325951132	2025-04-24T14:32:21.938+00:...
...	...	...	...	...	...	...

loan scdtype1

▶ ✓ 11:51 AM (<1s)

68

```
%sql
use catalog hive_metastore
```

OK

▶ ✓ 11:51 AM (6s)

69

```
%sql
CREATE TABLE IF NOT EXISTS loan_databricks (
  loan_id int,
  customer_id int,
  loan_amount int,
  interest_rate int,
  loan_term int,
  hashkey BIGINT,
  createddate TIMESTAMP,
  createdby STRING,
  updateby STRING,
  updateddate TIMESTAMP
)
USING DELTA
LOCATION "/mnt/task/gold/loan_databricks";
```

OK

▶ ✓ 11:51 AM (<1s)

70

Python   

```
src_path = '/mnt/task/silverdatabricks/loans/'
dbutils.fs.ls(src_path)
```

```
[FileInfo(path='dbfs:/mnt/task/silverdatabricks/loans/_SUCCESS', name='_SUCCESS', size=0, modificationTime=1745509910000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loans/_committed_6229936791619473724', name='_committed_6229936791619473724', size=233, modificationTime=1745509909000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loans/_committed_8026898336602437240', name='_committed_8026898336602437240', size=124, modificationTime=1745504588000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loans/_committed_vacuum7261754158519325731', name='_committed_vacuum7261754158519325731', size=96, modificationTime=1745509910000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loans/_started_6229936791619473724', name='_started_6229936791619473724', size=0, modificationTime=1745509908000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/loans/part-00000-tid-6229936791619473724-091d2a4d-7a11-4682-b9e3-4b4911c21fa3-26-1-c000.snappy.parquet', name='part-00000-tid-6229936791619473724-091d2a4d-7a11-4682-b9e3-4b4911c21fa3-26-1-c000.snappy.parquet', size=2831, modificationTime=1745509908000)]
```

▶ ✓ 11:51 AM (<1s)

71

```
tgt_path = '/mnt/task/gold/loan_databricks'
dbutils.fs.ls(tgt_path)
```

```
[FileInfo(path='dbfs:/mnt/task/gold/loan_databricks/_delta_log/', name='_delta_log/', size=0, modificationTime=1745505158000),
 FileInfo(path='dbfs:/mnt/task/gold/loan_databricks/part-00000-ea452a5a-74c9-460f-bc0e-491d281c858c-c000.snappy.parquet', name='part-00000-ea452a5a-74c9-460f-bc0e-491d281c858c-c000.snappy.parquet', size=4825, modificationTime=1745505340000)]
```

▶

▼

✓

11:51 AM (<1s)

72

Python

✦

⌵

⋮

```
df_loan = spark.read.format("parquet").load(src_path)
display(df_loan)
```

▶ (2) Spark Jobs

▶ df_loan: pyspark.sql.dataframe.DataFrame = [loan_id: integer, customer_id: integer ... 3 more fields]

Table ▼

+

🔍

🔼

📄

🗑️

	^{1.2} ₃ loan_id	^{1.2} ₃ customer_id	^{1.2} loan_amount	^{1.2} interest_rate	^{1.2} ₃ loan_term	
1	1	45	10000.5	5.5	36	
2	2	12	20000.75	4.5	48	
3	3	78	15000	6	60	
4	4	34	30000.25	3.5	24	
5	5	56	25000	5	36	
6	6	23	17500.5	4	48	
7	7	89	22500.75	6.5	60	
8	8	67	27500	3	24	
9	9	14	32500.25	5.5	36	
10	10	92	37500.5	4.5	48	

▶

▼

✓

11:51 AM (<1s)

73

Python

✦

⌵

⋮

```
df4_hask = df_loan.withColumn("hashkey", crc32(concat(*df_loan.columns)))
display(df4_hask)
```

▶ (1) Spark Jobs

▶ df4_hask: pyspark.sql.dataframe.DataFrame = [loan_id: integer, customer_id: integer ... 4 more fields]

Table ▼

+

🔍

🔼

📄

🗑️

	^{1.2} ₃ loan_id	^{1.2} ₃ customer_id	^{1.2} loan_amount	^{1.2} interest_rate	^{1.2} ₃ loan_term	^{1.2} ₃ hashkey	
1	1	45	10000.5	5.5	36	2502041691	
2	2	12	20000.75	4.5	48	3355992476	
3	3	78	15000	6	60	4060872917	
4	4	34	30000.25	3.5	24	2847880128	
5	5	56	25000	5	36	1867834730	
6	6	23	17500.5	4	48	172242294	
7	7	89	22500.75	6.5	60	1348082437	
8	8	67	27500	3	24	4106075248	
9	9	14	32500.25	5.5	36	3981072152	

▶ ▼ ✓ 11:51 AM (1s)

74

Python

```
%python
from delta.tables import DeltaTable
dtable4 = DeltaTable.forPath(spark, tgt_path)
display(dtable4.toDF())
```

- ▶ (2) Spark Jobs

Table  



	1 ² ₃ loan_id	1 ² ₃ customer_id	1 ² ₃ loan_amount	1 ² ₃ interest_rate	1 ² ₃ loan_term	1 ² ₃ hashkey	📅 createddate
1	1	45	10000	5	36	2502041691	2025-04-24T14:
2	2	12	20000	4	48	3355992476	2025-04-24T14:
3	3	78	15000	6	60	4060872917	2025-04-24T14:
4	4	34	30000	3	24	2847880128	2025-04-24T14:
5	5	56	25000	5	36	1867834730	2025-04-24T14:
6	6	23	17500	4	48	172242294	2025-04-24T14:
7	7	89	22500	6	60	1348082437	2025-04-24T14:

11:51 AM (1s)

75

```
df_src4 = df4_hash.alias("src").join(dtable4.toDF().alias("tgt"), ((col("src.loan_id") == col("tgt.loan_id")) & (col("src.hashkey") == col("tgt.hashkey"))), "anti").select("src.*")
display(df_src4)
```

- ▶ (2) Spark Jobs

```
df_src4: pyspark.sql.dataframe.DataFrame = [loan_id: integer, customer_id: integer ... 4 more fields]
```

Table 

	1^2_3 loan_id	1^2_3 customer_id	1.2 loan_amount	1.2 interest_rate	1^2_3 loan_term	1^2_3 hashkey
--	-----------------	---------------------	-----------------	-------------------	-------------------	-----------------

No rows returned

```
%python
from pyspark.sql.functions import col, lit, current_timestamp

dtable4.alias("tgt").merge(
    df_src4.alias("src"), col("src.loan_id") == col("tgt.loan_id"))\
    .whenMatchedUpdate(set={
        "loan_id": col("src.loan_id"),
        "customer_id": col("src.customer_id"),
        "loan_amount": col("src.loan_amount"),
        "interest_rate": col("src.interest_rate"),
        "loan_term": col("src.loan_term"),
        "hashkey": col("src.hashkey"),
        "updateddate": current_timestamp(),
        "updateby": lit("databricks-update")
    })\
    .whenNotMatchedInsert(values={
        "loan_id": col("src.loan_id"),
        "customer_id": col("src.customer_id"),
        "loan_amount": col("src.loan_amount"),
        "interest_rate": col("src.interest_rate"),
        "loan_term": col("src.loan_term"),
        "hashkey": col("src.hashkey"),
        "createddate": current_timestamp(),
        "updateddate": current_timestamp(),
        "createdby": lit("databricks"),
        "updateby": lit("databricks")
    })
).execute()
```

11:51 AM (1s)

77

```
%sql
select * from hive_metastore.default.loan_databricks
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [loan_id: integer, customer_id: integer ... 8 more fields]

Table +

Q Y I □

	¹ ₃ loan_id	¹ ₃ customer_id	¹ ₃ loan_amount	¹ ₃ interest_rate	¹ ₃ loan_term	¹ ₃ hashkey	createddate
1	1	45	10000	5	36	2502041691	2025-04-24T14:
2	2	12	20000	4	48	3355992476	2025-04-24T14:
3	3	78	15000	6	60	4060872917	2025-04-24T14:
4	4	34	30000	3	24	2847880128	2025-04-24T14:
5	5	56	25000	5	36	1867834730	2025-04-24T14:
6	6	23	17500	4	48	172242294	2025-04-24T14:
7	7	89	22500	6	60	1348082437	2025-04-24T14:

transactions scdtype

```
11:51 AM (<1s) 79

%sql
use catalog hive_metastore
```

OK

```
11:51 AM (<1s) 80

%sql
CREATE TABLE IF NOT EXISTS transcation_databricks (
  transaction_id int,
  account_id int,
  transaction_date date,
  transaction_amount int,
  transaction_type string,
  hashkey BIGINT,
  createddate TIMESTAMP,
  createdby STRING,
  updateby STRING,
  updateddate TIMESTAMP
)
USING DELTA
LOCATION "/mnt/task/gold/transcation_databricks";
```

```
11:51 AM (<1s) 81

src_path = '/mnt/task/silverdatabricks/transactions/'
dbutils.fs.ls(src_path)
```

```
[FileInfo(path='dbfs:/mnt/task/silverdatabricks/transactions/_SUCCESS', name='_SUCCESS', size=0, modificationTime=1745509913000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/transactions/_committed_2867773204639543754', name='_committed_2867773204639543754', size=232, modificationTime=1745509913000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/transactions/_committed_413543053534269710', name='_committed_413543053534269710', size=123, modificationTime=1745504603000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/transactions/_committed_vacuum5902226319674452412', name='_committed_vacuum5902226319674452412', size=95, modificationTime=1745509914000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/transactions/_started_2867773204639543754', name='_started_2867773204639543754', size=0, modificationTime=1745509912000),
 FileInfo(path='dbfs:/mnt/task/silverdatabricks/transactions/part-00000-tid-2867773204639543754-0ef0718a-91fd-4768-8384-5a51b430531f-32-1-c000.snappy.parquet', name='part-00000-tid-2867773204639543754-0ef0718a-91fd-4768-8384-5a51b430531f-32-1-c000.snappy.parquet', size=3163, modificationTime=1745509913000)]
```

```
11:51 AM (<1s) 82

tgt_path = '/mnt/task/gold/transcation_databricks'
dbutils.fs.ls(tgt_path)
```

```
[FileInfo(path='dbfs:/mnt/task/gold/transcation_databricks/_delta_log/', name='_delta_log/', size=0, modificationTime=1745505374000),
 FileInfo(path='dbfs:/mnt/task/gold/transcation_databricks/part-00000-1adafa05-070d-4529-a6d9-2c7ffcb4877b-c000.snappy.parquet', name='part-00000-1adafa05-070d-4529-a6d9-2c7ffcb4877b-c000.snappy.parquet', size=5322, modificationTime=1745505438000)]
```

▶ 11:51 AM (1s)83Python

df_transcation = spark.read.format("parquet").load(src_path)
display(df_transcation)

▶ (2) Spark Jobs

df_transcation: pyspark.sql.dataframe.DataFrame = [transaction_id: integer, account_id: integer ... 3 more fields]

Table +

	transaction_id	account_id	transaction_date	transaction_amount	transaction_type
1	1	45	2024-01-01	100.5	Deposit
2	2	12	2024-01-02	200.75	Withdrawal
3	3	78	2024-01-03	150	Deposit
4	4	34	2024-01-04	300.25	Withdrawal
5	5	56	2024-01-05	250	Deposit
6	6	23	2024-01-06	175	Withdrawal
7	7	89	2024-01-07	225.5	Deposit

▶ 11:51 AM (<1s)84

df5_hask = df_transcation.withColumn("hashkey", crc32(concat(*df_transcation.columns)))
display(df5_hask)

▶ (1) Spark Jobs

df5_hask: pyspark.sql.dataframe.DataFrame = [transaction_id: integer, account_id: integer ... 4 more fields]

Table +

	transaction_id	account_id	transaction_date	transaction_amount	transaction_type	hashkey
1	1	45	2024-01-01	100.5	Deposit	284663
2	2	12	2024-01-02	200.75	Withdrawal	46328
3	3	78	2024-01-03	150	Deposit	274580
4	4	34	2024-01-04	300.25	Withdrawal	275467
5	5	56	2024-01-05	250	Deposit	72569
6	6	23	2024-01-06	175	Withdrawal	216869
7	7	89	2024-01-07	225.5	Deposit	27940
8	8	67	2024-01-08	275.75	Withdrawal	142639


```
%python
from delta.tables import DeltaTable
dtable5 = DeltaTable.forPath(spark, tgt_path)
display(dtable5.toDF())
```

(2) Spark Jobs

	transaction_id	account_id	transaction_date	transaction_amount	transaction_type	hashkey
1	1	45	2024-01-01	100	Deposit	284663
2	2	12	2024-01-02	200	Withdrawal	46328
3	3	78	2024-01-03	150	Deposit	274586
4	4	34	2024-01-04	300	Withdrawal	275467
5	5	56	2024-01-05	250	Deposit	72565
6	6	23	2024-01-06	175	Withdrawal	216865
7	7	89	2024-01-07	225	Deposit	27946
8	8	67	2024-01-08	275	Withdrawal	142635
9	9	14	2024-01-09	325	Deposit	306043

- ▶ (2) Spark Jobs

	1 ² ₃ transaction_id	1 ² ₃ account_id	📅 transaction_date	1 ² ₃ transaction_amount	A ^B _C transaction_type	1 ² ₃ hashkey
1	1	45	2024-01-01	100	Deposit	284663
2	2	12	2024-01-02	200	Withdrawal	46328
3	3	78	2024-01-03	150	Deposit	274580
4	4	34	2024-01-04	300	Withdrawal	275467
5	5	56	2024-01-05	250	Deposit	72565
6	6	23	2024-01-06	175	Withdrawal	216865
7	7	89	2024-01-07	225	Deposit	27940
8	8	67	2024-01-08	275	Withdrawal	142635
9	9	14	2024-01-09	325	Deposit	306043

```
df_src5 = df5_hask.alias("src").join(dtable5.toDF().alias("tgt"), ((col("src.transaction_id") == col("tgt.transaction_id")) & (col("src.hashkey") == col("tgt.hashkey"))), "anti").select("src.*")
display(df_src5)
```

(2) Spark Jobs

df_src5: pyspark.sql.dataframe.DataFrame = [transaction_id: integer, account_id: integer ... 4 more fields]

transaction_id	account_id	transaction_date	transaction_amount	transaction_type	hashkey
No rows returned					

- ▶ (2) Spark Jobs

Table

1_3 transaction_id	1_3 account_id	 transaction_date	1.2 transaction_amount	A_C^B transaction_type	1_3 hashkey
No rows returned					

11:51 AM (4s)

87

Python

```
%python
from pyspark.sql.functions import col, lit, current_timestamp

dtable5.alias("tgt").merge(
    df_src5.alias("src"), col("src.transaction_id") == col("tgt.transaction_id"))\
        .whenMatchedUpdate(set={
            "transaction_id": col("src.transaction_id"),
            "account_id": col("src.account_id"),
            "transaction_date": col("src.transaction_date"),
            "transaction_amount": col("src.transaction_amount"),
            "transaction_type": col("src.transaction_type"),
            "hashkey": col("src.hashkey"),
            "updatedate": current_timestamp(),
            "updateby": lit("databricks-update")
        })\
        .whenNotMatchedInsert(values={
            "transaction_id": col("src.transaction_id"),
            "account_id": col("src.account_id"),
            "transaction_date": col("src.transaction_date"),
            "transaction_amount": col("src.transaction_amount"),
            "transaction_type": col("src.transaction_type"),
            "hashkey": col("src.hashkey"),
            "createddate": current_timestamp(),
            "updatedate": current_timestamp(),
            "createdby": lit("databricks"),
            "updateby": lit("databricks")
        })
).execute()
```

11:51 AM (1s)

88

```
%sql
select * from hive_metastore.default.transaction_databricks
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [transaction_id: integer, account_id: integer ... 8 more fields]

	transaction_id	account_id	transaction_date	transaction_amount	transaction_type	hashkey
1	1	45	2024-01-01	100	Deposit	284665
2	2	12	2024-01-02	200	Withdrawal	46328
3	3	78	2024-01-03	150	Deposit	274580
4	4	34	2024-01-04	300	Withdrawal	275467
5	5	56	2024-01-05	250	Deposit	72569

COMBINE ALL THE FILES

combine all the five files into delta file format

▶ ✓ 02:44 PM (<1s) 90

```
# List the files
files = dbutils.fs.ls("/mnt/task/silverdatabricks/accounts/")

# Find the part-xxxxx file
output_file = [f.path for f in files if f.name.startswith("part-")][0]

# Move and rename
dbutils.fs.mv(output_file,("/mnt/task/silverdatabricks/accounts/accounts.parquet"))
```

True

▶ ▾ ✓ 02:44 PM (<1s) 91 Python ✨ ⌵ ⋮

```
df1 = spark.read.format("parquet").load("/mnt/task/silverdatabricks/accounts/accounts.parquet").alias("df1")
display(df1)
```

▶ (2) Spark Jobs

▶ 📄 df1: pyspark.sql.dataframe.DataFrame = [account_id: integer, customer_id: integer ... 2 more fields]

Table ▾ + 🔍 ⚙️ 📄

	¹ ₃ account_id	¹ ₃ customer_id	^A _C account_type	1.2 balance
1	1	45	Savings	1000.5
2	2	12	Checking	2500.75
3	3	78	Savings	1500
4	4	34	Checking	3000.25
5	5	56	Savings	500
6	6	23	Checking	1200.5
7	7	89	Savings	800.75
8	8	67	Checking	2200

```
02:44 PM (<1s) 92

# List the files
files = dbutils.fs.ls("/mnt/task/silverdatabricks/customers/")

# Find the part-xxxxx file
output_file = [f.path for f in files if f.name.startswith("part-")][0]

# Move and rename
dbutils.fs.mv(output_file, "/mnt/task/silverdatabricks/customers/customers.parquet"))
```

True

```
02:44 PM (<1s) 93

df2 = spark.read.format("parquet").load("/mnt/task/silverdatabricks/customers/customers.parquet").alias("df2")
display(df2)
```

(2) Spark Jobs

df2: pyspark.sql.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

	customer_id	first_name	last_name	address	city	state	zip
1	1	John	Doe	123 Elm St	Toronto	ON	M4B1B3
2	2	Jane	Smith	456 Maple Ave	Ottawa	ON	K1A0B1
3	3	Michael	Johnson	789 Oak Dr	Montreal	QC	H1A1A1
4	4	Emily	Davis	101 Pine Rd	Calgary	AB	T2A0A1
5	5	David	Wilson	202 Birch Blvd	Vancouver	BC	V5K0A1

```
02:44 PM (<1s) 94

# List the files
files = dbutils.fs.ls("/mnt/task/silverdatabricks/loan_payments/")

# Find the part-xxxxx file
output_file = [f.path for f in files if f.name.startswith("part-")][0]

# Move and rename
dbutils.fs.mv(output_file, "/mnt/task/silverdatabricks/loan_payments/loan_payments.parquet"))
```

True

```
02:44 PM (<1s) 95

df3 = spark.read.format("parquet").load("/mnt/task/silverdatabricks/loan_payments/loan_payments.parquet").alias("df3")
display(df3)
```

(2) Spark Jobs

df3: pyspark.sql.dataframe.DataFrame = [payment_id: integer, loan_id: integer ... 2 more fields]

	payment_id	loan_id	payment_date	payment_amount
1	1	45	2024-01-01	100
2	2	23	2024-01-02	150
3	3	67	2024-01-03	200
4	4	89	2024-01-04	250
5	5	12	2024-01-05	300

```
▶ ✓ 02:44 PM (<1s) 96 Python ✨ ⌵ ⋮

# List the files
files = dbutils.fs.ls("/mnt/task/silverdatabricks/loans/")

# Find the part-xxxxx file
output_file = [f.path for f in files if f.name.startswith("part-")][0]

# Move and rename
dbutils.fs.mv(output_file,("/mnt/task/silverdatabricks/loans/loans.parquet"))
```

True

```
▶ ✓ 02:44 PM (<1s) 97

df4 = spark.read.format("parquet").load("/mnt/task/silverdatabricks/loans/loans.parquet").alias("df4")
display(df4)
```

▶ (2) Spark Jobs

▶ df4: pyspark.sql.dataframe.DataFrame = [loan_id: integer, customer_id: integer ... 3 more fields]

	1 ² loan_id	1 ² customer_id	1.2 loan_amount	1.2 interest_rate	1 ² loan_term
1	1	45	10000.5	5.5	36
2	2	12	20000.75	4.5	48
3	3	78	15000	6	60
4	4	34	30000.25	3.5	24

```
▶ ✓ 02:44 PM (<1s) 98 Python ✨ ⌵ ⋮

# List the files
files = dbutils.fs.ls("/mnt/task/silverdatabricks/transactions/")

# Find the part-xxxxx file
output_file = [f.path for f in files if f.name.startswith("part-")][0]

# Move and rename
dbutils.fs.mv(output_file,("/mnt/task/silverdatabricks/transactions/transactions.parquet"))
```

True

```
▶ ✓ 02:44 PM (<1s) 99

df5 = spark.read.format("parquet").load("/mnt/task/silverdatabricks/transactions/transactions.parquet").alias("df5")
display(df5)
```

▶ (2) Spark Jobs

▶ df5: pyspark.sql.dataframe.DataFrame = [transaction_id: integer, account_id: integer ... 3 more fields]

	1 ² transaction_id	1 ² account_id	📅 transaction_date	1.2 transaction_amount	1 ² transaction_type
1	1	45	2024-01-01	100.5	Deposit
2	2	12	2024-01-02	200.75	Withdrawal
3	3	78	2024-01-03	150	Deposit
4	4	34	2024-01-04	300.25	Withdrawal
5	5	56	2024-01-05	250	Deposit

▶ ✓ 02:44 PM (<1s) 100 Python

```
#Combine the files
df_combined = df1.join(df2, df2.customer_id == df2.customer_id, "inner").join(df4, df2.customer_id == df4.customer_id,
"inner").join(df3, df4.loan_id == df3.loan_id, "inner").join(df5, df1.account_id == df5.account_id, "inner")
```

▶ df_combined: pyspark.sql.dataframe.DataFrame = [account_id: integer, customer_id: integer ... 23 more fields]

▶ ✓ 02:44 PM (<1s) 101

```
#Select the required columns and modify the datatype
df_selected_columns = df_combined.select(
    col("df1.account_id").alias("Account_ID").cast("int"),
    col("df2.customer_id").alias("Customer_ID").cast("int"),
    col("df4.loan_id").alias("Loan_ID").cast("int"),
    col("df3.payment_id").alias("Payment_ID").cast("int"),
    col("df5.transaction_id").alias("Transaction_ID").cast("int"),
    col("df1.balance").alias("Account_Balance").cast("double"),
    col("df3.payment_amount").alias("Payment_Amount").cast("double"),
    col("df4.loan_amount").alias("Loan_Amount").cast("double"),
    col("df5.transaction_amount").alias("Transaction_Amount").cast("double"),
    col("df3.payment_date").alias("Payment_Date").cast("date"),
    col("df5.transaction_date").alias("Transaction_Date").cast("date")
)
```

▶ df_selected_columns: pyspark.sql.dataframe.DataFrame = [Account_ID: integer, Customer_ID: integer ... 9 more fields]

▶ ✓ 02:44 PM (<1s) 102

```
%sql
USE CATALOG hive_metastore;
CREATE TABLE IF NOT EXISTS Combined
(
    Account_ID INT,
    Customer_ID INT,
    Loan_ID INT,
    Payment_ID INT,
    Transaction_ID INT,
    Account_Balance DOUBLE,
    Payment_Amount DOUBLE,
    Loan_Amount DOUBLE,
    Transaction_Amount DOUBLE,
    Payment_Date DATE,
    Transaction_Date DATE
)
USING DELTA
LOCATION '/mnt/task/silverdatabricks/combined/Combined';
```

OK

▶ ✓ 02:44 PM (4s) 103

```
#Write the dataframe to the delta table
df_selected_columns.write.format("delta").mode("overwrite").save('/mnt/task/silverdatabricks/combined/Combined')
```

▶ (14) Spark Jobs

02:44 PM (1s) 104

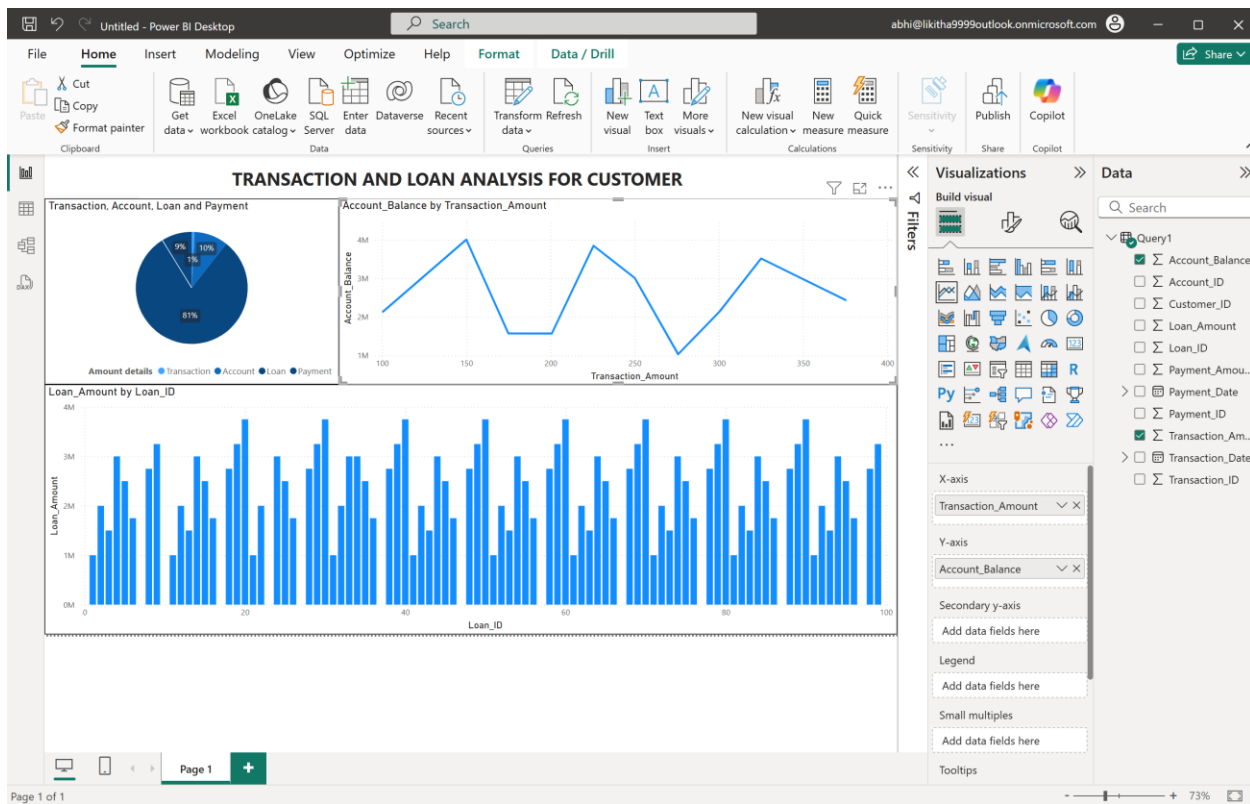
```
%sql
Select * from hive_metastore.default.combined
```

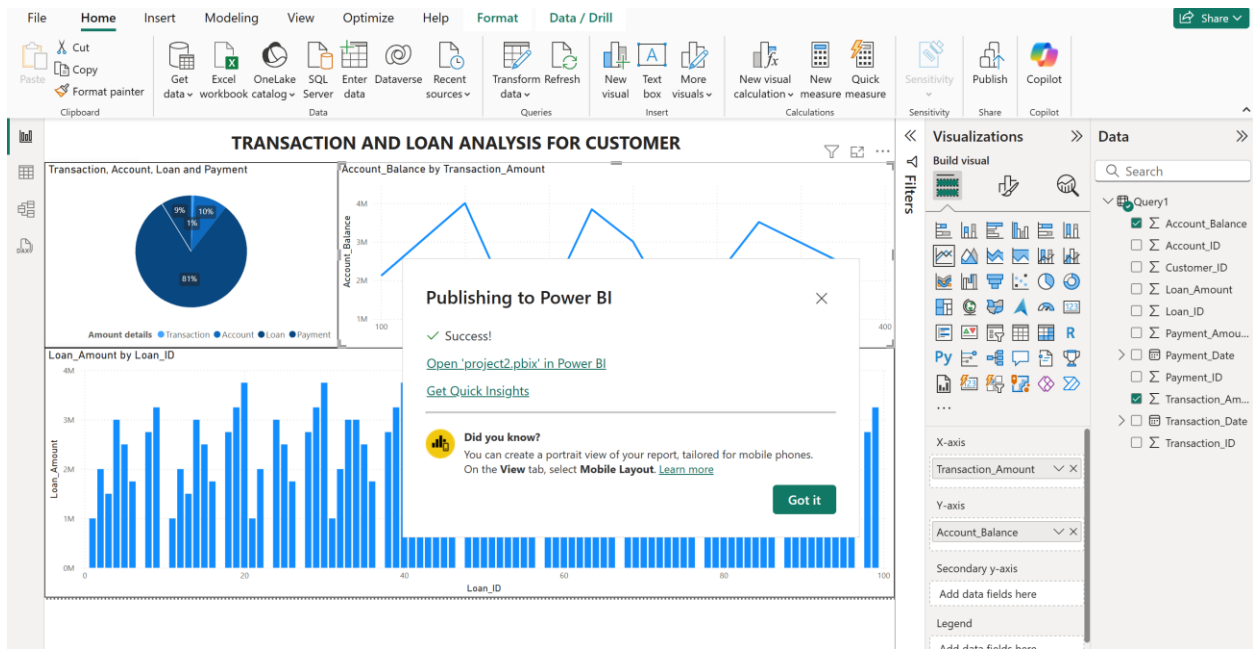
(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [Account_ID: integer, Customer_ID: integer ... 9 more fields]

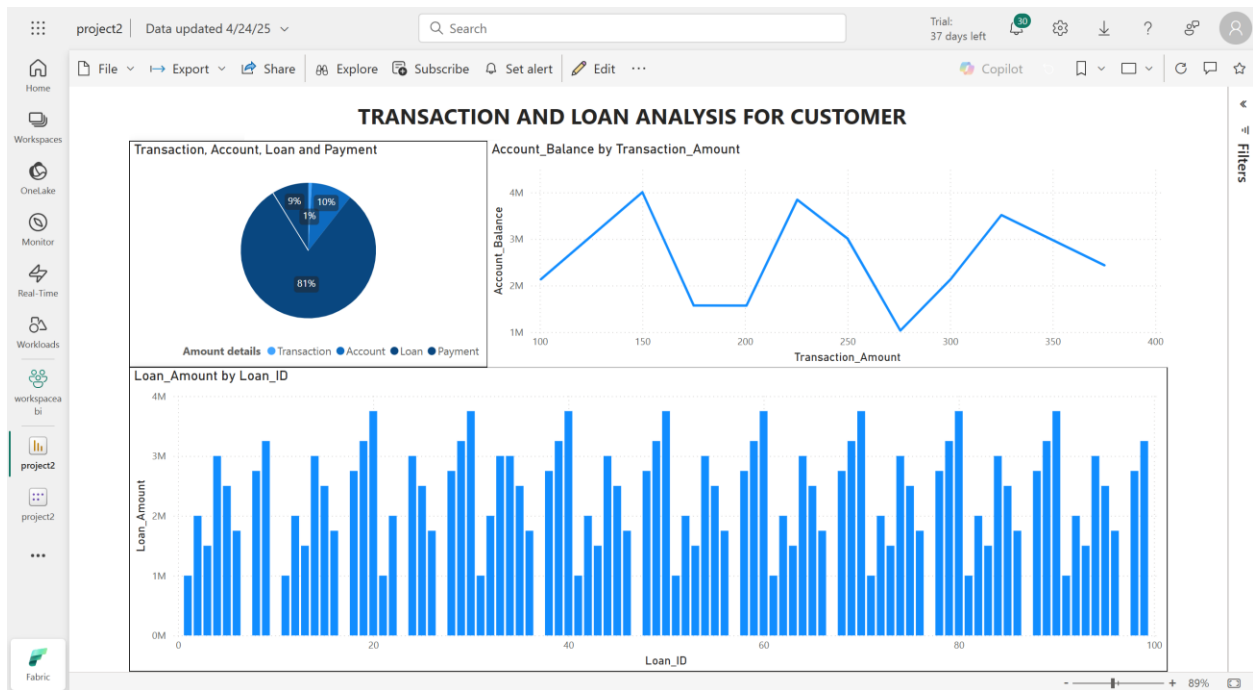
	Account_ID	Customer_ID	Loan_ID	Payment_ID	Transaction_ID	Account_Balance	
1	45	1	88	17	1	300.25	
2	45	2	82	71	1	300.25	
3	45	3	11	10	1	300.25	
4	45	4	78	8	1	300.25	
5	45	5	18	47	1	300.25	
6	45	6	48	77	1	300.25	
7	45	7	28	57	1	300.25	
8	45	8	68	97	1	300.25	
9	45	9	32	21	1	300.25	
10	45	10	52	41	1	300.25	

GENERATE REPORTS IN POWER BI





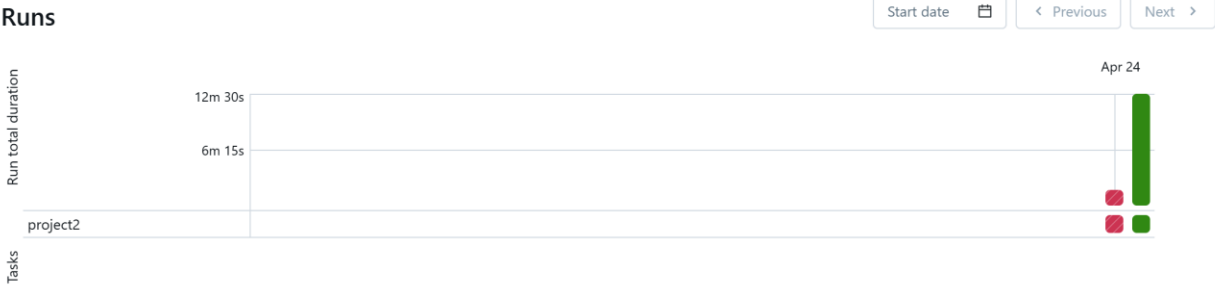
PUBLISHED IN FABRIC WORKSPACE



CREATED AND RUN THE JOB SUCCESSFULLY

RunsTasks

Show side panel



Go to the latest successful run

Cancel runs

Start time	Run ID	Launched	Duration	Spark	Status	Error code	Run parameters	
Apr 24, 2025, 02:59 PM	108852038638...	By scheduler	12m 30s	Spark UI / Logs / Metrics	✔ Succeeded			